# AicoreLink — Fábrica Admin Redesign + Cleanups + Production Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

**Goal:** Modernize the AicoreLink admin panel (shell + `/fabrica` dashboard + review panel) to the AiCoreMed visual language, remove the Semilla plan and one marketing line, then execute the landing visual redesign and ship everything to production on the VPS via Dokploy.

**Architecture:** The admin panel is a Next.js App Router route group `(admin)` whose pages are client components that fetch `/api/*`. We extract the chrome into a client `AdminShell` (dark navy collapsible sidebar with `lucide-react` icons + active state, sticky header), add a small set of presentation primitives under `components/ui/`, and restyle `/fabrica` and `/fabrica/[siteId]` in place — **keeping every existing `data-testid`** so the current unit + e2e suites stay green. Cleanups are surgical copy/test edits. The landing redesign follows its own approved plan. The final stage deploys `main` to the VPS through Dokploy.

**Tech Stack:** Next.js 14.2 App Router · TypeScript 5.5 · Tailwind CSS 3.4 · `lucide-react` (already a dependency) · vitest + RTL · Playwright.

**Spec / references:**

- AiCoreMed admin visual language (read-only reference): `D:\OPENCODE\consultorio-medico\dashboard\` — `app/globals.css` (HSL tokens, `--radius: 0.75rem`, card shadows), `tailwind.config.ts` (shadcn token mapping, `hoverable` variant, `fade-in-up` / `scale-in` / `slide-in-right` keyframes, `ease-out` `cubic-bezier(0.16,1,0.3,1)`), `components/layout/sidebar.tsx`.
- Landing redesign plan (Stage D): `docs/superpowers/plans/2026-10-13-aicorelink-landing-visual-redesign.md`.
- Master build plan: `docs/superpowers/plans/2026-10-12-aicorelink-web-master-build-plan.md`.

## Global Constraints

- Work on branch `feat/landing-v2`; production deploys `main`. All commands run from `apps/dashboard`.
- **No new runtime dependencies.** `lucide-react` and `clsx` / `tailwind-merge` (`cn` in `@/lib/utils`) are already available. Do **not** add shadcn/ui, Radix or recharts.
- **Preserve every existing `data-testid`** relied on by `e2e/factory.spec.ts`: `fabrica-table`, `fabrica-row`, `filter-status`, `filter-rubro`, `filter-search`, `bulk-approve`, `fabrica-error`, `panel-approve`, `panel-reject`, `panel-unpublish`. They must exist and be visible at **all three** e2e viewports (375/768/1440).
- Keep `<table>` markup for the site list (wrapped in `overflow-x-auto`); do not replace it with a card list, or the mobile e2e breaks.
- UI copy: **Chilean Spanish tuteo**, never voseo. Code, identifiers, comments, commits: **English**, conventional commits, **no AI attribution**.
- Admin panel is internal: keep `metadata.robots = { index: false, follow: false }` and `export const dynamic = "force-dynamic"` in `(admin)/layout.tsx`.

- Re-read the reference files before styling; do not invent tokens — admin uses Tailwind's built-in `slate` / `white` scales plus the design tokens that already exist (`navy`, `brand`, `teal`, `amber`).

## Progress

- [ ] **Stage A** — Cleanups (Semilla removal, verticals note removal)
- [ ] **Stage B** — Admin shell redesign
- [ ] **Stage C** — Fábrica dashboard + review panel redesign
- [ ] **Stage D** — Landing visual redesign (9 tasks from the approved plan)
- [ ] **Stage E** — Production deploy via Dokploy

## Review Focus

1. `data-testid` **survival across viewports.** The dashboards tests run at 375/768/1440; a responsive redesign is the classic way to delete a testid from one breakpoint. → pinned in Task C3 Step 1/4.
2. **Empty/zero states.** A brand-new install has zero sites; the KPI row and table must render a real empty state, not `NaN` or a blank card. → pinned in Task C1/C3.
3. **Failed fetches.** `/api/admin/sites` can fail; the page must show the `fabrica-error` banner and keep the filters usable. → pinned in Task C3.
4. **Sidebar on mobile.** The drawer must be reachable and closable (Escape / backdrop) without trapping focus or scrolling the body. → pinned in Task B1.
5. **Semilla removal completeness.** The plan likes to creep back in via tests, e2e strings, the comparison table and the redesigned pricing section. → pinned in Task A1 + Task D updates.

---

## Stage A — Cleanups

### Task A1: Remove the Semilla plan and the verticals note

**Files:**

- Modify: `apps/dashboard/lib/marketing/copy.ts` (`pricing.plans` 4 → 3, remove `verticals.note`)
- Modify: `apps/dashboard/components/marketing/verticals-section.tsx` (drop the note paragraph)
- Modify: `apps/dashboard/__tests__/pricing-section.test.tsx`
- Modify: `apps/dashboard/__tests__/verticals-section.test.tsx`
- Modify: `apps/dashboard/__tests__/marketing-copy.test.ts`
- Modify: `apps/dashboard/e2e/responsive.spec.ts`
- Modify: `docs/superpowers/plans/2026-10-13-aicorelink-landing-visual-redesign.md` (Task 4: 4 → 3 plans; drop the `#seed` aside)

**Interfaces:**

- Consumes: nothing.
- Produces: `copy.pricing.plans` length **3** (`starter`, `growth` highlighted/bestSeller, `elite`); the `Semilla` entry and the `LandingCopy` type member for `verticals.note` are gone.

- [ ] **Step 1: Remove Semilla from the copy**

In `copy.ts`, delete `plans[0]` (`id: "semilla"`). Keep `title`, `subtitle`, `note` and the three remaining plans unchanged (all `+ IVA`).

- [ ] **Step 2: Remove the verticals note**

Delete the `note` field from `copy.verticals` and its type declaration. In `verticals-section.tsx`, delete the `<p …>{copy.verticals.note}</p>` line. Leave `title`, `subtitle`, `ctaTemplate` intact.

- [ ] **Step 3: Update the unit tests**

- `pricing-section.test.tsx`: delete the `getByText("sin costo")` assertion; keep `getAllByText("+ IVA")` (length **3**), the highlighted-is-`growth` assertion, both badges, and the CTA-count-===`plans.length` assertion. Any `plans[0]` reference now means `starter` — verify the CTA-name assertion still matches.
- `verticals-section.test.tsx`: delete the `getByText(copy.verticals.note)` assertion.
- `marketing-copy.test.ts`: delete the assertion containing `"catálogo"`.

- [ ] **Step 4: Update the e2e**

In `responsive.spec.ts`: change `#planes article[data-plan]` count from **4** to **3**; delete the assertion for the exact text `$0`; keep `$19.990`, `$49.990`, `$99.990`.

- [ ] **Step 5: Update the landing redesign plan**

In `docs/superpowers/plans/2026-10-13-aicorelink-landing-visual-redesign.md` Task 4, change "keep **4** `article[data-plan]` (incl. Semilla)" to **3**, remove the Semilla/`#seed` requirement from the comparison-table step, and adjust the "Must keep" bullet that counts 4 plans.

- [ ] **Step 6: Run the tests**

Run: `npm run test -- __tests__/pricing-section.test.tsx __tests__/verticals-section.test.tsx __tests__/marketing-copy.test.ts` Expected: PASS.

- [ ] **Step 7: Commit**

```
git add apps/dashboard/lib/marketing/copy.ts apps/dashboard/components/marketing/verticals-section.
git commit -m "chore(marketing): drop the Semilla plan and the verticals disclaimer"
```

---

## Stage B — Admin shell redesign

### Task B1: `AdminShell` — dark collapsible sidebar with icons + sticky header

**Files:**

- Create: `apps/dashboard/components/admin/admin-shell.tsx` (`"use client"`)

- Create: `apps/dashboard/components/admin/admin-nav.ts` (nav item definitions)
- Modify: `apps/dashboard/app/(admin)/layout.tsx`
- Modify: `apps/dashboard/tailwind.config.ts` (only if a shadow/animation utility is missing)
- Test: `apps/dashboard/__tests__/admin-shell.test.tsx` (new)

**Interfaces:**

- Consumes: `lucide-react`, `cn` from `@/lib/utils`, `Badge` from `@/components/ui/badge`.
- Produces:
  - `admin-nav.ts` → `export interface AdminNavItem { href: string; label: string; icon: LucideIcon; disabled?: boolean }` and `export const ADMIN_NAV: AdminNavItem[]` with, in order: `/` **Menú principal** (`Home`), `/tenants` **Tenants** (`Building2`), `/fabrica` **Fábrica** (`Factory`), `/metricas` **Métricas** (`BarChart3`, `disabled: true`), `/embudo` **Embudo** (`Filter`), `/infraestructura` **Infraestructura** (`Server`, `disabled: true`).
  - `admin-shell.tsx` → `export function AdminShell({ children }: { children: React.ReactNode })`. Renders: `lg`-fixed sidebar (`bg-slate-900 text-white`, `w-64` ↔ `w-16` collapsed, persisted in `localStorage` under `aicorelink.admin.sidebar`), brand block, nav from `ADMIN_NAV` with `usePathname()` active styling (`bg-white/10 text-white` + a `teal` left rail; inactive `text-slate-300 hover:bg-white/5 hover:text-white`), sign-out at the bottom; a mobile drawer (backdrop + `Escape` close + body-scroll lock) toggled by a sticky header button with `aria-expanded`/`aria-controls`; and a sticky header showing the active page title + breadcrumb. `<main>` keeps `bg-slate-50 p-4 lg:p-8`.
  - `(admin)/layout.tsx` stays a Server Component exporting `dynamic`/`metadata`, and renders `<Providers><AdminShell>{children}</AdminShell></Providers>`.

- [ ] **Step 1: Write the failing test**

Create `apps/dashboard/__tests__/admin-shell.test.tsx`:

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { describe, expect, it, vi } from "vitest";
import { AdminShell } from "@/components/admin/admin-shell";
import { ADMIN_NAV } from "@/components/admin/admin-nav";

vi.mock("next/navigation", () => ({ usePathname: () => "/fabrica" }));

describe("AdminShell", () => {
  it("renders every nav entry exactly once", () => {
    render(<AdminShell><p>contenido</p></AdminShell>);
    ADMIN_NAV.forEach((item) => {
      expect(screen.getByRole("link", { name: item.label })).toHaveAttribute("href", item.href);
    });
  });
  it("marks the active route", () => {
    render(<AdminShell><p>contenido</p></AdminShell>);
    const active = screen.getByRole("link", { name: "Fábrica" });
    expect(active.getAttribute("aria-current")).toBe("page");
  });
  it("renders each nav entry with an icon", () => {
    const { container } = render(<AdminShell><p>contenido</p></AdminShell>);
    expect(container.querySelectorAll("nav svg").length).toBeGreaterThanOrEqual(ADMIN_NAV.length);
  });
});
```

(If `@testing-library/user-event` is not installed, drop the import and the interaction assertions — keep the render queries.)

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/admin-shell.test.tsx` Expected: FAIL — `Failed to resolve import "@/components/admin/admin-shell"`.

- [ ] **Step 3: Implement `admin-nav.ts` and `admin-shell.tsx`**

Build the shell as described in Interfaces. Use `aria-current="page"` on the active link and `aria-disabled` + `pointer-events-none opacity-50` for disabled entries. Collapse state must not be read during SSR (guard with a `useEffect`).

- [ ] **Step 4: Wire it into the layout**

Replace the inline `<aside>`/`<nav>` in `(admin)/layout.tsx` with `<AdminShell>`, keeping `dynamic` and `metadata` exports untouched.

- [ ] **Step 5: Run the tests and the build**

Run: `npm run test -- __tests__/admin-shell.test.tsx` → PASS. Run: `npm run typecheck` → clean. `npm run build` → succeeds.

- [ ] **Step 6: Commit**

```
git add apps/dashboard/components/admin apps/dashboard/app/\(admin\)/layout.tsx apps/dashboard/__te
git commit -m "feat(admin): add icon sidebar shell with active state and mobile drawer"
```

---

## Stage C — Fábrica dashboard + review panel redesign

### Task C1: Add aggregate stats to the admin sites API

**Files:**

- Modify: `apps/dashboard/app/api/admin/sites/route.ts`
- Modify: `apps/dashboard/__tests__/app/api/admin/sites.test.ts`

**Interfaces:**

- Consumes: `asAdmin` (tenant-context), prisma.
- Produces: the `GET` response gains a sibling key `stats: { total: number; review: number; published: number; draft: number }` computed **unfiltered** (independent of the `status`/`rubro`/`q` query params), while `sites` stays filtered as today.

- [ ] **Step 1: Extend the test**

Add to `__tests__/app/api/admin/sites.test.ts`: with a filter applied, `stats.total` still counts **all** sites, and `stats.review`/`published`/`draft` reflect the unfiltered status distribution.

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test -- __tests__/app/api/admin/sites.test.ts` Expected: FAIL (`stats` undefined).

- [ ] **Step 3: Implement the stats**

In the route, run the filtered `findMany` for `sites` and a `groupBy({ by: ["status"], _count: true })` for the counts; map the four keys above (defaulting missing statuses to 0).

- [ ] **Step 4: Run the test**

Run: `npm run test -- __tests__/app/api/admin/sites.test.ts` Expected: PASS.

- [ ] **Step 5: Commit**

```
git add apps/dashboard/app/api/admin/sites/route.ts apps/dashboard/__tests__/app/api/admin/sites.te
git commit -m "feat(admin): expose unfiltered site stats from the admin sites API"
```

### Task C2: Presentation primitives

**Files:**

- Create: `apps/dashboard/components/ui/stat-card.tsx`
- Create: `apps/dashboard/components/ui/skeleton.tsx`
- Create: `apps/dashboard/components/ui/empty-state.tsx`
- Create: `apps/dashboard/components/ui/page-header.tsx`
- Create: `apps/dashboard/components/ui/initials-avatar.tsx`
- Test: `apps/dashboard/__tests__/ui-admin-primitives.test.tsx` (new)

**Interfaces:**

- Consumes: `cn`, `lucide-react`.
- Produces:
  - `StatCard({ label: string; value: string | number; icon: LucideIcon; tone?: "neutral" | "brand" | "amber" | "teal"; hint?: string })`
  - `Skeleton({ className?: string })`
  - `EmptyState({ icon: LucideIcon; title: string; description?: string; action?: React.ReactNode })`
  - `PageHeader({ title: string; description?: string; icon?: LucideIcon; actions?: React.ReactNode })`
  - `InitialsAvatar({ name: string; className?: string })` — deterministic initials + a colour derived from the name.

- [ ] **Step 1: Write the failing test** asserting: `StatCard` renders label + value + a `tone`-specific class; `EmptyState` renders title/description/action; `InitialsAvatar` renders the expected initials (e.g. `"Barbería El Corte"` → `"BE"`).
- [ ] **Step 2: Run it to verify it fails** (`Failed to resolve import`).
- [ ] **Step 3: Implement the five primitives**
- [ ] **Step 4: Run the tests** → PASS.
- [ ] **Step 5: Commit**

```
git add apps/dashboard/components/ui/stat-card.tsx apps/dashboard/components/ui/skeleton.tsx apps/d
git commit -m "feat(ui): add stat card, skeleton, empty state, page header and avatar primitives"
```

### Task C3: Redesign the `/fabrica` dashboard

**Files:**

- Modify: `apps/dashboard/app/(admin)/fabrica/page.tsx`
- Test: `apps/dashboard/__tests__/app/api/admin/sites.test.ts` (already covers the API)
- Test: `apps/dashboard/e2e/factory.spec.ts` (extend; do not weaken)

- Test: `apps/dashboard/__tests__/fabrica-page.test.tsx` (new, if the page can be rendered with mocked fetch)

**Interfaces:**

- Consumes: `AdminSite` (unchanged), `stats` from `GET /api/admin/sites`, the Task C2 primitives.
- Produces: same component contract; **same testids**.
- [ ] **Step 1: Write the failing assertions**

Extend `e2e/factory.spec.ts` (dashboard describe): after load, the four KPI cards are visible and the values match the mocked fixture; the empty-state is shown when the API returns `sites: []`; the `fabrica-error` banner appears when the API returns 500 **and** the filters are still enabled. Keep every existing testid assertion.

- [ ] **Step 2: Run it to verify it fails**

Run: `npm run test:e2e -- e2e/factory.spec.ts` (with the server on 3111) Expected: FAIL (KPI cards missing).

- [ ] **Step 3: Rebuild the page**

`PageHeader` (icon `Factory`) + KPI row of four `StatCard`s (Total / En revisión / Publicados / Borradores, from `stats`) + the filter `Card` (icon-labelled selects kept as `<select data-testid="filter-status|filter-rubro">`, search with a `Search` icon keeping `data-testid="filter-search"`, a "Limpiar" button) + bulk-approve button (`data-testid="bulk-approve"`, label `Aprobar seleccionados (n)`). Table **unchanged in structure**: `<div className="overflow-x-auto"><table data-testid="fabrica-table">` with `tr[data-testid="fabrica-row"]`; restyle cells only — `InitialsAvatar` for the site, rubro/plan with icons, status `Badge`, relative published date, and a `Revisar` link plus an external-link icon when `status === "published"`. Loading → `Skeleton` rows; empty → `EmptyState` **inside the table** (keep the `fabrica-table` element present); fetch failure → `fabrica-error` banner and leave the filters enabled.

- [ ] **Step 4: Run the e2e at all three viewports**

Run: `npm run build` → `npm run start -- --port 3111` → `npm run test:e2e -- e2e/factory.spec.ts` Expected: **all green** (dashboard + review funnel × 375/768/1440).

- [ ] **Step 5: Commit**

```
git add apps/dashboard/app/\(admin\)/fabrica/page.tsx apps/dashboard/e2e/factory.spec.ts
git commit -m "feat(admin): redesign the fabrika dashboard with KPIs, icons and states"
```

## Task C4: Redesign the `/fabrica/[siteId]` review panel

**Files:**

- Modify: `apps/dashboard/app/(admin)/fabrica/[siteId]/site-review-panel.tsx`
- Modify: `apps/dashboard/app/(admin)/fabrica/[siteId]/page.tsx` (only for the header/breadcrumb)
- Test: `apps/dashboard/e2e/factory.spec.ts` (the review-funnel describe must keep passing)

**Interfaces:**

- Consumes: Task C2 primitives, `renderBlocks`.
- Produces: same props (`{ site, tenantId, onRefresh }`), same testids, same checklist logic (the `useMemo`-derived `checklistResults`).

- [ ] **Step 1: Restyle without moving the testids**

Keep `data-testid="panel-approve"`, `panel-reject`, `panel-unpublish` on the same interactive elements, keep the checklist gate text `"Completa todos los ítems del checklist para habilitar Aprobar"`, and keep the three-column layout collapsing to a single column below `lg` (this is what makes the mobile e2e pass). Modernize: `PageHeader` with breadcrumb, `Card`s with the soft shadow scale, status `Badge` with icon, `Skeleton` while loading, icons from `lucide-react`.

- [ ] **Step 2: Run the review-funnel e2e**

Run: `npm run test:e2e -- e2e/factory.spec.ts` Expected: **all green**.

- [ ] **Step 3: Full gates**

Run: `npm run test` → all green (baseline 487 + new). `npm run typecheck` → clean. `npm run lint` → 0 errors.

- [ ] **Step 4: Commit**

```
git add apps/dashboard/app/\(admin\)/fabrica/\[siteId\]
git commit -m "feat(admin): modernize the site review panel"
```

---

## Stage D — Landing visual redesign

Execute `docs/superpowers/plans/2026-10-13-aicorelink-landing-visual-redesign.md` **in full** (Tasks 1-9), with the Task 4 amendment from Task A1 Step 5 (3 plans, no Semilla). Announce: "I'm using the executing-plans skill to implement the landing redesign plan."

- [ ] **Task D1 (plan Task 1)** — tokens + self-hosted fonts + section shell
- [ ] **Task D2 (plan Task 2)** — hero
- [ ] **Task D3 (plan Task 3)** — dolores
- [ ] **Task D4 (plan Task 4)** — planes (3 plans)
- [ ] **Task D5 (plan Task 5)** — FAQ (12)
- [ ] **Task D6 (plan Task 6)** — CTA
- [ ] **Task D7 (plan Task 7)** — footer
- [ ] **Task D8 (plan Task 8)** — assembly + SEO
- [ ] **Task D9 (plan Task 9)** — e2e + Lighthouse

---

## Stage E — Production deploy via Dokploy

### Task E1: VPS environment and static-files runtime

**Files:**

- Modify: `.env.example` / deployment docs if present (documentation only)
- VPS: Dokploy project for AicoreLink

**Interfaces:**

- Produces: the env vars the E2.6/E2.7 publish flow needs on the VPS: `FACTORY_INTERNAL_API_KEY`, `N8N_FACTORY_PUBLISH_WEBHOOK_URL`, `N8N_FACTORY_PUBLISH_WEBHOOK_SECRET`, `STATIC_SITES_PORT`; the Traefik dynamic config dir + ACME dir and the static-files container listening on `3001`.

- [ ] **Step 1: Capture the current Dokploy env** for the AicoreLink app (`dokploy.aicorebots.com`, project → app → Environment) and diff against what the code reads.
- [ ] **Step 2: Add the missing variables** in Dokploy (never echo secrets into the transcript).
- [ ] **Step 3: Verify the static-files container / Traefik routing** used by the publish flow.
- [ ] **Step 4: Document the outcome** in the plan/ledger (names only, no values).

### Task E2: Ship to production

- [ ] **Step 1: Final gates on the branch** `npm run test` · `npm run typecheck` · `npm run lint` · `npm run build` · `npm run test:e2e` — all green.
- [ ] **Step 2: Finish the branch** with `superpowers:finishing-a-development-branch` (merge `feat/landing-v2` → `main`).
- [ ] **Step 3: Push** `main` and let Dokploy build/deploy (or trigger the deploy from the Dokploy UI).
- [ ] **Step 4: Verify production** `https://aicorelink.aicorebots.com` → HTTP 200; the landing shows the redesigned hero and **3** plans; `/fabrica` renders for an admin session; the console has zero errors.
- [ ] **Step 5: Lighthouse in production** — no regression vs the documented baseline (A11y 100 / BP 100 / SEO 100, Perf ≈ 91-96).

---

## Self-Review

**1. Coverage.** Semilla removal → A1 (+ D4 amendment). Verticals note removal → A1. Fábrica redesign → B1 (shell), C1-C4 (stats, primitives, dashboard, review panel). Landing redesign → D. Dokploy/VPS/production → E. Progress checklist → the `## Progress` section above (kept updated as tasks complete).

**2. Step scan.** Each step runs one command or edits one named file.

**3. Type consistency.** `AdminNavItem` / `ADMIN_NAV` (B1) are consumed only by `AdminShell`; `stats` (C1) is consumed by C3; the C2 primitive signatures are used verbatim in C3/C4.

**4. Review Focus.** testid survival → C3 Step 1/4 (e2e at 3 viewports). Zero/empty states → C1 + C3 Step 3 (`EmptyState` inside the table). Failed fetches → C3 Step 3. Mobile sidebar → B1 Step 3 (Escape + backdrop + scroll lock). Semilla completeness → A1 + D4.

**5. Proportion.** Four stages; the landing redesign is delegated to its own detailed plan rather than duplicated here.